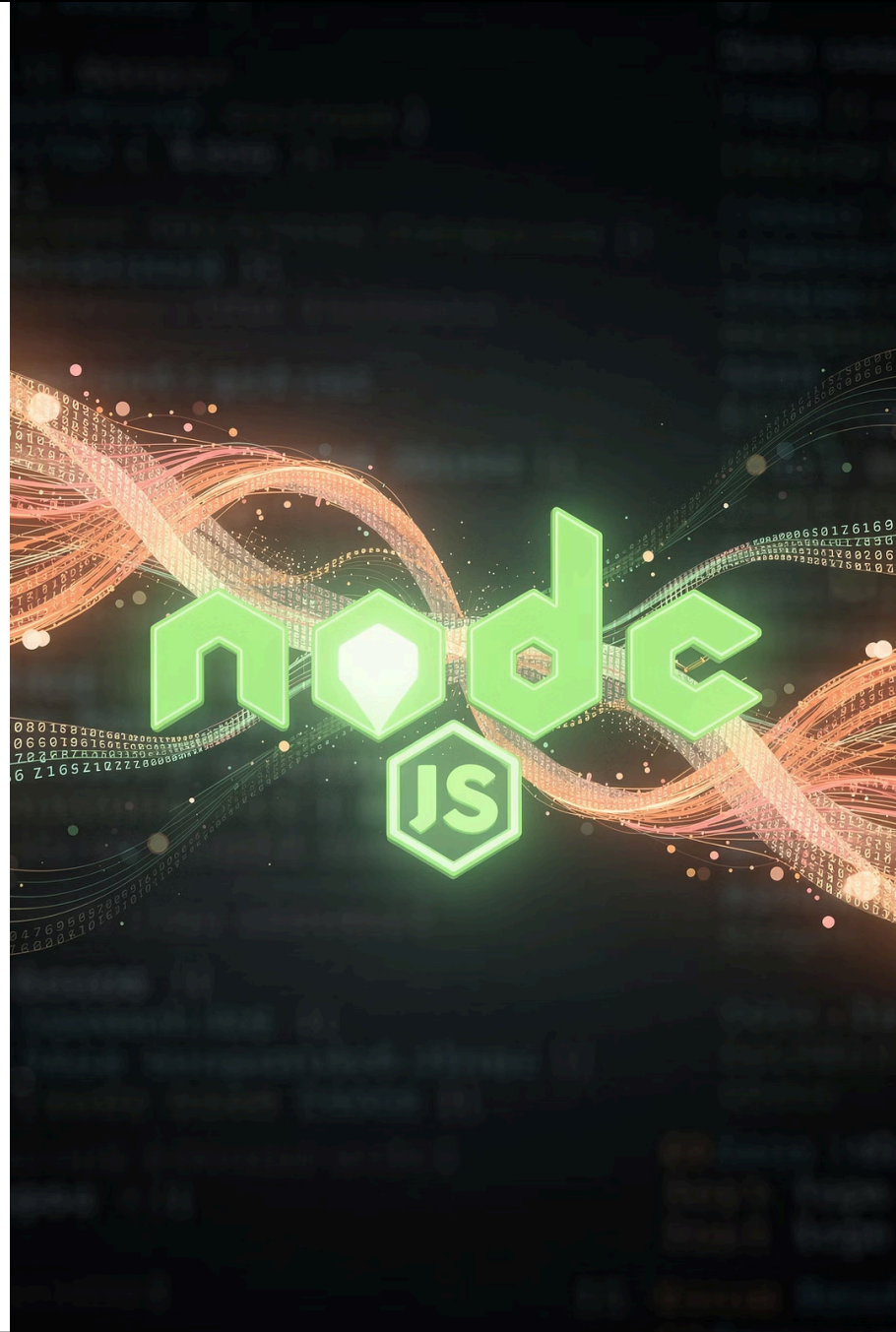
Node.js

Fast, scalable, and event-driven JavaScript runtime built on Chrome's V8 engine.



Node.js

JavaScript Runtime Environment

It is used for server side programming.

*Node.js is not a language, library or framework.

```
Welcome to Node.js v16.13.0.
Type ".help" for more information.
> ls
Uncaught ReferenceError: ls is not defined
> pwd
Uncaught ReferenceError: pwd is not defined
> 1+2
3
> console.log("hello");
hello
undefined
> "hello"+" shradha"
'hello shradha'
> window
Uncaught ReferenceError: window is not defined
> global
<ref *1> Object [global] { global: [Circular *1], clearInterval: [F
```

Process

process : This object provides information about, and control over, the current Node.js process.

process.argv : returns an array containing the command-line arguments passed when the Node.js process was launched.

Exporting Files

module.exports

requiring files

require( ) a built-in function to include external modules that exist in separate files.

module.exports a special object

```
JS math.js > obj > PI
const sum = (a, b) => a + b
const mul = (a, b) => a * b
const g = 9.8;
const PI = 3.14;

let obj = {
  sum: sum,
  mul: mul,
  g: g,
  PI: PI,
};

module.exports = obj;
```

```
const math = require("./math");

console.log(math.sum(2, 2));
console.log(math.PI);
```

```
module.exports.sum = (a, b) => a + b;
module.exports.mul = (a, b) => a * b;
module.exports.g = 9.8;
module.exports.PI = 3.14;
```

Exporting Directories

```
module.exports = {
  name: "apple",
  color: "red",
};
```

```
const apple = require("./apple");
const banana = require("./banana");
const orange = require("./orange");

let fruits = [apple, banana, orange];

module.exports = fruits;
```

```
const info = require("./Fruits");

console.log(info);
```

NPM

Example: npm install figlet

usually it creates 3 folders: node_modules, package.json, package-lock.json

Installing Packages

node_modules The node_modules folder contains every installed dependency for your project.

package-lock.json It records the exact version of every installed dependency, including its sub-dependencies and their versions.

package.json

The package. json file contains descriptive and functional **metadata** about a project, such as a name, version, and dependencies

npm init

Local v/s Global

npm install -g <- package name ->

npm link <- package name ->

- `my-package` → package you are building
- `my-app` → app where you want to use it

Inside package folder:

```bash
cd my-package
npm link
```

This makes your package available as a local linked package. npm Docs

Step 2

Inside app folder:

```bash
cd my-app
npm link my-package
```

require vs import

import { sum } from "./math.js"

We can't selectively load only the pieces we need with require but with import, we can selectively load only the pieces we need, which can save memory.

Loading is synchronous for 'require' but can be asynchronous for 'import'.

1. Default import

file: `math.js`

JavaScript
```
export default function add(a, b) {
  return a + b;
}
```

file: `app.js`

JavaScript
```
import add from "./math.js";

console.log(add(2, 3)); // 5
```

2. Named import

file: `math.js`

JavaScript
```
export function add(a, b) {
  return a + b;
}

export function sub(a, b) {
  return a - b;
}
```

file: `app.js`

JavaScript
```
import { add, sub } from "./math.js";

console.log(add(5, 2)); // 7
console.log(sub(5, 2)); // 3
```

Import Everything

JavaScript
```
import * as math from "./math.js";

console.log(math.add(4, 1)); // 5
console.log(math.sub(4, 1)); // 3
```